Imprecise Probabilistic Programming, Precisely

From an abstract graded monad to concrete BDD inference

Jack Liell-Cock Sam Staton

University of Oxford

July 24, 2026

Motivation

Probabilistic programming

- Model probabilistic processes
- Uncertainty with *known* probabilities

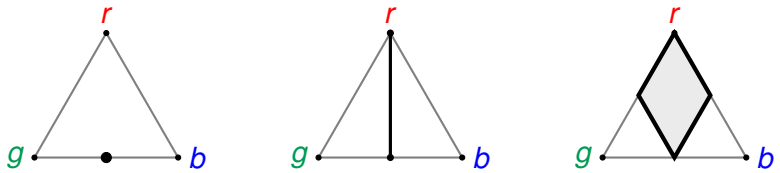
Imprecise probabilistic programming

- Model probabilistic *and Knightian* processes
- Some probabilities are *unknown*

Inference under model uncertainty

Background: imprecise probability

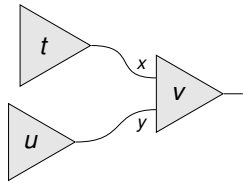
- **Probability** = a single point in the simplex
- **Imprecise probability** = a convex set of points, a *credal set*
- Quantities of interest: *lower* and *upper* probability



Background: CP is not commutative

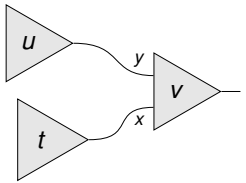
The *convex powerset* (CP) monad models sets of distributions, but **it is not commutative**:

```
x <- t ; y <- u ; v
```



$\neq$ in CP

```
y <- u ; x <- t ; v
```



The fix (earlier work)

Give each Knightian source a **name**; grade the monad by its *set of names*.
Tracking names restores commutativity and standard program equivalences.

This talk

1. A Haskell-embedded **graded-monad DSL** for an imprecise PPL.
   Type-level names track each Knightian source.
2. Imprecise inference needs no new compilation infrastructure.
   Probabilistic *and* Knightian choices compile to BDD variables.
3. One compilation, **three** inference methods.
   Exact enumeration, gradient descent, and interval-arithmetic bounds.

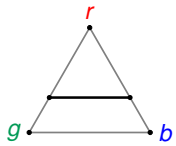
Core is ≈1,200 lines of Haskell. Standard BDD + WMC machinery.
Available at: `github.com/jacklc3/imp`

The graded monad, in Haskell

`Imp g a`: an imprecise computation of `a` with Knightian names `g`
(semantically a graded monad: $\texttt{Imp g a} = (2^g \to D(a))$)

```
data Ball = Red | Green | Blue

ellsberg :: Imp '["split"] Ball
ellsberg = Imp.do
  isRed   ← flip (1/3)
  isGreen ← knight @"split"
  Imp.return $
    if isRed then Red
    else if isGreen then Green
    else Blue
```



Ellsberg's urn: contains 30 red balls and 60 green or blue balls.

- `flip p :: Imp '[] Bool` – ordinary biased coin
- `knight @"n" :: Imp '["n"] Bool` – coin of *unknown* bias, named `"n"`
- **bind:** unions the grades enforcing disjointness at compile time: no spurious correlations
- **`Imp.do`:** `QualifiedDo` for ergonomic notation

Names live only in types and are **erased at runtime**

The behaviour of names

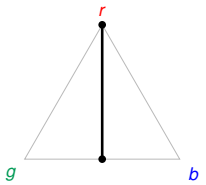
Program

```
dependent :: Imp '["a1"] Ball
dependent = Imp.do
  x ← flip 0.5
  if x then Imp.do
    y ← knight @"a1"
    Imp.return (if y then Red else Green)
  else Imp.do
    y ← knight @"a1"
    Imp.return (if y then Red else Blue)
```

Compilation/Inference

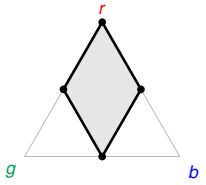
⇒

Credal set




```
independent :: Imp '["a1","a2"] Ball
independent = Imp.do
  x ← flip 0.5
  if x then Imp.do
    y ← knight @"a1"
    Imp.return (if y then Red else Green)
  else Imp.do
    y ← knight @"a2"    -- a2, not a1
    Imp.return (if y then Red else Blue)
```

⇒

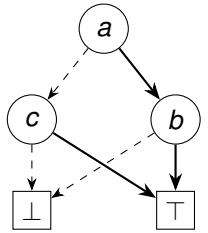


Binary decision diagrams (BDDs)

A **BDD** is an encoding of a Boolean function.

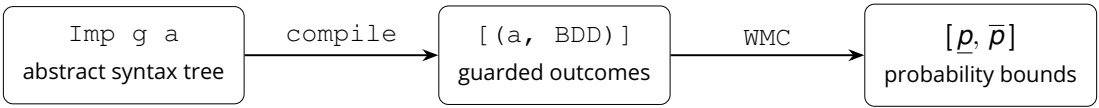
- Each *node* tests one variable
- **Solid** edge for true, **dashed** edge for false
- The two leaves are $\bot$ (false) and $\top$ (true)
- **Reduced** & **ordered**: equal subgraphs merged and redundant tests dropped

`if` *a* `then` *b* `else` *c*



A BDD is just the Boolean formula; attaching *weights* to the variables creates probabilities.

The pipeline



Compilation by structural recursion:

- **`flip p`:** a fresh *weighted* BDD variable (weight p)
- **`knight @"n"`:** an *unweighted* BDD variable; same name = same variable

Key idea: `flip` and `knight` produce **the same kind of object**

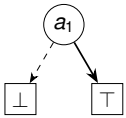
This is *exactly* the pipeline of a *precise* discrete language (e.g. Dice).

Dependent vs. independent

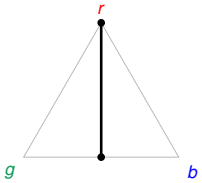
Program

```
dependent :: Imp '["a1"] Ball
dependent = Imp.do
  x ← flip 0.5
  if x then Imp.do
    y ← knight @"a1"
    Imp.return (if y then Red else Green)
  else Imp.do
    y ← knight @"a1"
    Imp.return (if y then Red else Blue)
```

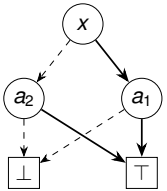
BDD (for Red)

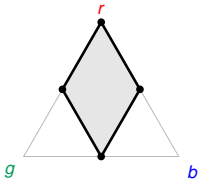


Credal set




```
independent :: Imp '["a1","a2"] Ball
independent = Imp.do
  x ← flip 0.5
  if x then Imp.do
    y ← knight @"a1"
    Imp.return (if y then Red else Green)
  else Imp.do
    y ← knight @"a2"    -- a2, not a1
    Imp.return (if y then Red else Blue)
```





Inference: semiring-parametric weighted model counting (WMC)

One algorithm, **three** inference methods:

$$\mathrm{WMC}(\top) = 1, \quad \mathrm{WMC}(\bot) = 0, \quad \mathrm{WMC}\big(v \mapsto (l, h)\big) = w_v^{\bot} \cdot \mathrm{WMC}(l) \; + \; w_v^{\top} \cdot \mathrm{WMC}(h)$$

- **Real valued:** Enumerate all $2^{|g|}$ valuations of the Knightian variables
- **Dual numbers:** Forward-mode AD for gradient ascent over the credal set
- **Interval arithmetic:** A *sound*, but not *tight*, bound in one pass

Variable weight	**Real**	**Dual**	**Interval**
Probabilistic	p	$(p, \vec{0})$	$[p, p]$
Knightian	0 or 1	$(0.5, \vec{\delta}_n)$	$[0, 1]$

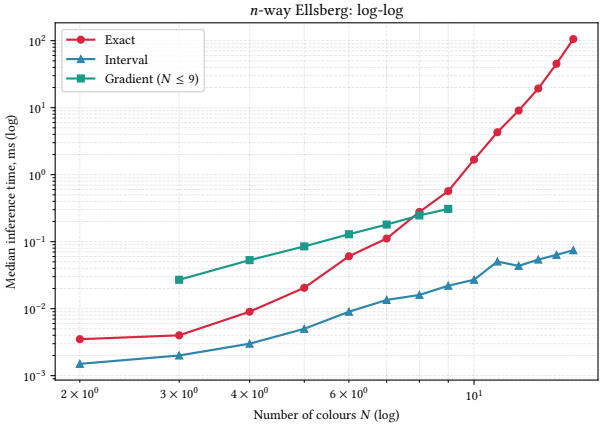
Performance on N-way Ellsberg urn

N-way Ellsberg urn:

- 1 ball has probability $\frac{1}{N}$
- Remaining have probability $[0, \frac{N-1}{N}]$
- Program is a BDD of size $O(N)$

Inference method:

- **Exact:** 2^{N-2} valuations
- **Gradient:** fixed number of passes
- **Interval:** single pass



Summary

A standard pipeline

- Type-level **names** restore commutativity and prevent interference: checked by GHC, erased at runtime
- Probabilistic *and* Knightian coins both compile to **BDD variables**: a knight is just a variable with a **free weight**
- One **semiring-parametric WMC** gives *three* inference methods

Imprecise probability requires no new compilation infrastructure.

Open directions: continuous distributions; SDDs / variable-order heuristics; iteration and infinite-dimensional Knightian structure.
Preprint available at: `arxiv.org/abs/2607.20801`

Code available at: `github.com/jacklc3/imp`